

TrafficOps

Автономный стенд безопасного canary-релиза через Kubernetes Gateway API

Назначение

Стенд объединяет развёртывание веб-сервиса, пробный выпуск v2, контролируемый сбой, диагностику и откат. Эксперт воспроизводит сценарий из публичного main командами `make bootstrap`, `make deploy`, `make verify` и `make verify-scenario`.

Репозиторий: <https://github.com/kkonstantin08/trafficops>

Архитектура и состав

Кластер: Ubuntu 24.04, kubeadm, Kubernetes 1.36.5, containerd 2.3.6 и Flannel 0.28.9; один узел. Поддерживаются AMD64/ARM64; чистая приёмочная среда - Ubuntu 24.04.4 AMD64, 4 CPU и около 3.8 GiB RAM.

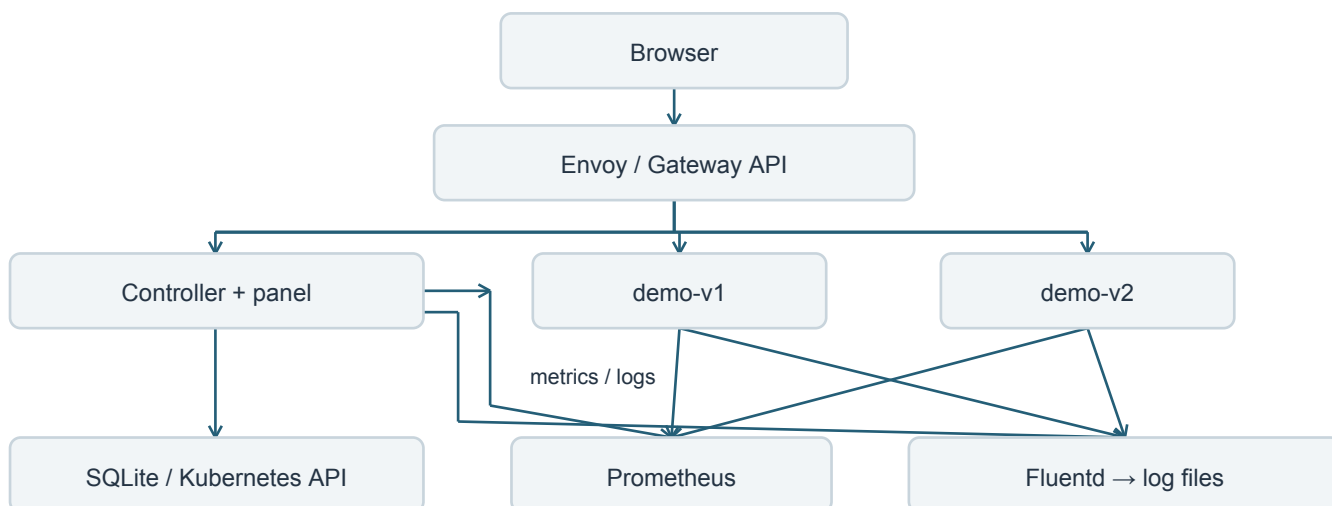
Вход: Envoy Gateway 1.9.1, GatewayClass, Gateway и HTTPRoute; NodePort 30080. Weighted backends распределяют трафик между demo-v1 и demo-v2; отдельный маршрут ведёт к FastAPI controller и статической HTML/CSS/JS панели.

Наблюдаемость: Prometheus развёрнут Kubernetes-манифестами; пять scrape jobs опрашивают demo-v1, demo-v2, Envoy proxy, node-exporter и kube-state-metrics с интервалом 10 секунд. Fluentd читает CRI access/error logs и сохраняет их в локальные persistent files; диагностика ищет уникальный маркер запроса.

Состояние: SQLite на local PV хранит операции, release state и sessions. Controller читает Prometheus и log files, сохраняет состояние и выполняет ограниченные действия через Kubernetes API.

Автоматизация: Makefile, shell scripts и Kubernetes manifests; Helm устанавливает Envoy Gateway. Основные версии, Helm checksums и platform-specific image digests закреплены.

Схема потоков



Пользовательские запросы проходят через Gateway; метрики и логи сохраняются отдельно от управляющего состояния.

Обязательная часть

Что реализовано	Зачем	Как проверено
kubeadm single-node; Ubuntu 24.04 AMD64/ARM64 bootstrap	Автономный запуск из публичного клона	Clean Ubuntu 24.04.4 AMD64: bootstrap PASS, node Ready, без ручных Kubernetes fixes
Python demo-v1/v2; /healthz, /metrics, structured logs	Проверяемый сервис двух версий	Gateway HTTP 200 и ожидаемый version JSON
Envoy Gateway; GatewayClass, Gateway, HTTPRoute; weighted backends	Маршрутизация и пробное обновление	Accepted/Programmed/ResolvedRefs и реальные HTTP-запросы через Gateway
Prometheus; пять scrape jobs	Оценка состояния и доли ошибок v2	Все targets up=1, свежие samples и реальные app counters
Fluentd; CRI parsing; persistent log files	Диагностика конкретного запроса	Маркер найден в точке назначения после Gateway request
make bootstrap / deploy / verify	Повторяемое развёртывание	Clean deploy и повторный запуск PASS; identity, storage и state сохранены

Реализованные улучшения

Canary: начальные веса 90/10, ручная настройка, завершение на v2, ручной и автоматический rollback. Политика оценивает только v2: полное окно 60 секунд после начала canary, минимум 30 запросов, 5xx >5%, проверки каждые 10 секунд и два превышения подряд; stale/unknown metrics блокируют продвижение. Успех rollback требует подтверждённого маршрута и ответа v1 через Gateway.

Сбой: контролируемые HTTP 500 на v2 демонстрируют automatic rollback. Механизм pod restart реализован; live recovery ранее подтверждён на ARM64, отдельно на новой AMD64 VM не повторялся.

Защита: PBKDF2 password hash, пароль вне Git, session cookie HttpOnly/SameSite=Strict, Origin/CSRF и bounded API inputs. HTTPRoute/Deployment/ConfigMap grants ограничены named resources; pod recovery имеет namespace-wide get/list/delete pods в trafficops. Фильтр demo pods в controller не является Kubernetes RBAC boundary.

CI: GitHub Actions проверяет 41 unit test, shell/JS, 37 manifest resources и сборку AMD64/ARM64. Main CI 37192333821 завершился success; публикация main и unauthenticated default clone подтверждены.

Результаты проверки

3 октября 2026: clean bootstrap, deploy/verify, healthy canary, v2 completion, manual rollback, faulted canary и automatic rollback - PASS; full scenario прошёл после исправления controller connection-idle timeout. Idempotence - PASS; HTTP security negative suite - 25 requests PASS; RBAC boundary suite - 34 checks PASS. Evidence и точные runtime revisions: docs/evidence/amd64-*.md; main promotion и public clone: docs/evidence/main-finalization-2026-10-04.md.

Инженерное ревью

Сильная техническая сторона

Единый воспроизводимый сценарий связывает deploy, canary, наблюдаемость, сбой, automatic rollback и проверку через реальный Gateway API. Решения опираются на фактические Prometheus samples и Fluentd logs, а восстановление подтверждается ответом стабильной версии сервиса.

Самое сложное решение и альтернативы

Для безопасного rollback отсутствие метрик нельзя считать нулём ошибок или смешивать старые samples с текущим canary: нужны полное окно, минимальная выборка и две последовательные проверки превышения. Один threshold check проще, но чувствительнее к шуму; фиксированный таймер без метрик хуже отражает состояние новой версии. Service mesh или Argo Rollouts дают больше возможностей, но для автономного одноузлового стенда выбран ограниченный controller с явно проверяемой политикой.

Развитие: multi-region / edge rollout

Следующий этап - постепенное обновление по площадкам и region-aware Gateway routing в нескольких кластерах. Потребуются multi-cluster control plane, внешние метрики и state, общая наблюдаемость.

Развитие: SLO-based rollback

Политику можно расширить latency, availability и traffic volume для SLO телеком-сервисов. Потребуются более длинная история метрик, настраиваемые политики и, при распределённой среде, Prometheus federation или remote storage.

Развитие: event / audit integration

Release и rollback incidents можно передавать во внешнюю NOC/incident system. Потребуются authenticated webhook или event bus, внешняя интеграция и централизованное состояние.

Ограничения

Один узел не обеспечивает HA; local PV не защищают от потери диска. HTTP demo не имеет TLS; pod delete RBAC действует на весь namespace trafficops, транзитивные APT packages не полностью pinned. Новая AMD64 visual browser verification намеренно не выполнялась; HTTP/API и runtime-пути проверены, визуальный рендер панели не заявляется.

Воспроизведение и доказательства

Порядок установки и проверки: README.md и docs/verification.md; статусы требований: docs/requirements/06-acceptance.md. Runtime evidence содержит условия, revisions и фактические результаты; паспорт описывает подтверждённую функциональную линию main, а не заменяет эти доказательства.